COP3530 – Assignment 4

Objective

Students will be able to develop code involving fundamental data structures, by implementing a solution to a problem that requires the use of hash tables and collision resolution using chaining.

Assignment Problem

In this exercise you are asked to implement a hash table that will store strings of characters. The following aspects are to be observed:

- *HashTable*, your main data structure, will have a size of 19.
- Collisions will be resolved using the **chaining** approach: strings that map to the same hash value are stored in a linked list.
- Hash function to be used is the polynomial hash function implemented in class on exercise *Prog23_01*. Use *p* (hash constant) equal to 31.
- To implement the linked list, use solution in *Prog13_01* (Canvas/Modules) modified to meet the UML diagram given below.
- Several files accompany this project description:
    - *Main.java*: a complete implementation of the `Main` class (tests your code using the Java keywords),
    - *HashTable.java*: the framework of the `HashTable` class,
    - *LinkedList.java*: the framework of the `LinkedList` class,
    - *output.pdf*: a file with the output generated by the `Main` class provided.
- The output of your program is expected to be the same as the output example provided.

Guidelines

- The assignment is to be completed individually or in teams of two students. The given problem is based on the content studied on hashing and collision resolution.

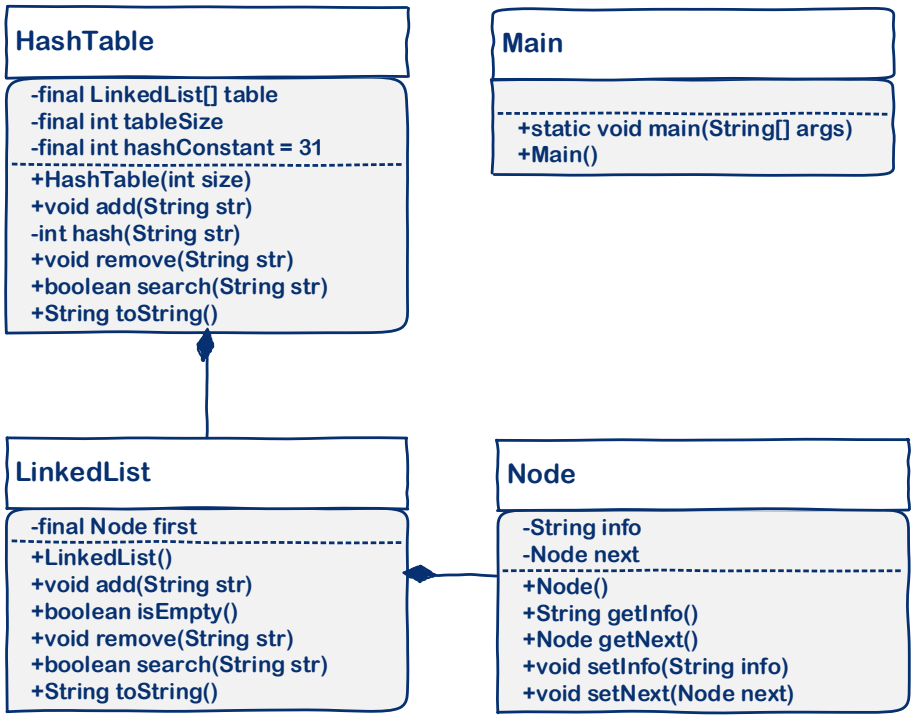
- You are allowed to use all of the code given in the lectures. In those cases, make sure you properly credit its source.

- Classes from the Java Collection Framework are not allowed for use in your implementation.

- Students are required to structure the code as indicated in the UML class diagram.

- Students are required to use the code framework given for each class, completing the methods according to the description provided in the Javadoc-style comment section of the methods.

UML class diagram



Deliverables

- A compressed folder, *PID Assignment 4* (e.g. *1234567 Assignment 4*), containing

  - files *LinkedList.java* and *HashTable.java*
  - A screenshot of the running program. Note: a screenshot is an image that shows 1) the IDE environment with code and 2) the output window. A partial view of the IDE with code is fine, the output window is to be displayed in its entirety. More than one image file might be required to capture everything.

- Include **only** the .java files mentioned above; do not include other files or folders generated by the IDE.

- Make sure you write your full name(s), Panther ID(s), and class section(s) in the @author field of each Java class.

Grading Rubric

The assignment is worth 115 points (out of 1000 total course points). Grade components:

<table>
<tr><th>Component</th><th>Points</th><th>Description</th></tr>
<tr><td>Submission</td><td>5</td><td>The student has submitted the project solution using the requirements for deliverables specified in the Deliverables section.</td></tr>
<tr><td>Organization</td><td>5</td><td>Code is expected to be neat, organized, and readable.</td></tr>
<tr><td>Content</td><td>105</td><td>
<table>
<tr><th>Deliverable</th><th>Points</th></tr>
<tr><td>HashTable class</td><td>50 pts</td></tr>
<tr><td>LinkedList class</td><td>50 pts</td></tr>
<tr><td>Screenshot(s)</td><td>5 pts</td></tr>
</table>
</td></tr>
</table>